Yojun Moon (문요준)

yojun313@postech.ac.kr github.com/yojun313 linkedin.com/in/yojun-moon yojun313.github.io

Education

POSTECH (Pohang University of Science and Technology) — *Mar 2023 – Present*

B.S. in Computer Science and Engineering

Work Experience

Samsung Fire & Marine Insurance — *Jul 2026 – Aug 2026*

AI Engineer Intern, AI Development Part

Developed computer vision modules for insurance claim data, connecting model training and evaluation to web-based processing services.

- **License plate masking:** Built a vehicle-to-plate detection pipeline with **99.0% recall** on the image evaluation set and **27+ fps** in the development GPU environment.
- **Document forgery detection:** Built FinForge with 6,000 original/forged images from eight generative AI models; fine-tuned TruFor and raised document-level test accuracy from **about 55% to 93%**.
- Evaluated degraded images and external data, then organized reports, code, and datasets for follow-up research.
- **Technologies:** Python, PyTorch, Ultralytics YOLO, OpenCV, FFmpeg, FastAPI.

Korean National Police University — FPEI (Future Public Safety Engineering Institute) — *Nov 2023 – Present*

Research System Developer

Developed research software for data collection, statistical analysis, network analysis, keyword signals, and document generation; extended desktop workflows to web services at knpu.re.kr/systems.

- Added a tool-using research assistant and evidence-based AI interpretation to the analysis services.
- Built asynchronous collection queues and shared job monitoring, LLM configuration, and usage tracking.
- Take sole responsibility for operating **two research servers**, including a **dual-A100 80GB** GPU server, MongoDB, local LLM serving, and Nginx.
- Received a Letter of Appreciation from the Korean National Police University for research software contributions (Apr 2024).
- **Technologies:** Python, FastAPI, MongoDB, Qt, Ubuntu Server, vLLM, Nginx, PM2.

POSTECH HAiV Lab — *Jan 2026 – Feb 2026*

Undergraduate Researcher

Contributed to ACCIDENTOR, a traffic-accident analysis framework using local vision-language models and embedding-based retrieval to support fault-ratio estimation from dashcam video.

- **Technologies:** Python, PyTorch, VLMs, embeddings, vector database.

Startup Team (founded by a POSTECH CSE alumnus) — *Feb 2025 – Jul 2025*

Backend Developer Intern

- Developed an SNS backend, web crawlers, and data analysis features; collaborated through Git and code review.
- Completed the 2025 I-Corps program, including three weeks of entrepreneurship training at UC Berkeley (KIC Tech Frontier Program).
- **Technologies:** TypeScript, NestJS, Python, MongoDB, Linux, Git.

Selected Projects

PCSS (POSTECH Computer Scientist Search)

Initial development: Jan 2025 – Mar 2026 · NameClassifier development ongoing

Commissioned faculty candidate search service for the POSTECH CSE department.

- Built DBLP ingestion, Grok-based name scoring, researcher profiles and rankings, monthly updates, and email subscriptions; reduced manual browsing from hours to queries taking seconds.
- Developing a Qwen3-0.6B LoRA name classifier using approximately 4.7 million name records; achieved **97.69% F1** on the benchmark at a 0.5 threshold.
- **Technologies:** Python, FastAPI, MongoDB, Grok API, PyTorch, Transformers, PEFT/LoRA, SQLite.

Commeet — *Aug 2026 – Present*

Meeting scheduling and consultation record service for the POSTECH CSE department, with **100+ cumulative users**.

- Used min-cost max-flow to maximize scheduled participants, with preference costs and regrouping heuristics.
- Added guest responses, email-domain verification, Google Calendar integration, and encrypted consultation records.
- **Technologies:** FastAPI, MongoDB, Motor, Jinja2, JavaScript, Google Calendar API, Fernet.

POPO — *Jan 2026 – Feb 2026*

Contributed to the backend of the campus web service operated by PoApper, POSTECH's student developer club.

- **Technologies:** TypeScript, NestJS.

Personal Projects

- **LecAI:** AI study assistant connecting slides, recordings, and board notes, with a chatbot for each slide; FastAPI, MongoDB, Whisper, LLM/VLM APIs.
- **UnivDash:** Web dashboard for coding agents and development tools; **I currently use it for all my development work**; FastAPI, WebSocket, tmux, PTY, PM2, Git.
- **vllm-manager:** CLI and web console for LLM deployment and GPU memory planning; FastAPI, Docker, vLLM, Hugging Face Hub.
- **Eng2Mouth:** Real-time AI voice conversations with post-call feedback; FastAPI, MongoDB, WebRTC, OpenAI Realtime API, Gemini Live API.

Activities

POSTECH Broadcasting Station (PBS) — *Mar 2023 – Dec 2025*

Member of the Programming & Production Department. Filmed, edited, and live-broadcast campus entertainment programs.

Programming Tutor — *Jan 2025 – Present*

Teaching Python and C to middle and high school students, including object-oriented programming, MongoDB, and FastAPI.

Skills

Languages	Python, C, C++, TypeScript, JavaScript
AI / Machine Learning	PyTorch, Transformers, PEFT/LoRA, YOLO, OpenCV, vLLM, LLM/VLM APIs
Backend / Data	FastAPI, NestJS, MongoDB, SQLite, REST APIs, WebSocket
Infrastructure / Tools	Linux, Docker, Nginx, PM2, Git, tmux, Qt, FFmpeg

---

1. Samsung Fire & Marine Insurance — Computer Vision Modules

Jul 2026 – Aug 2026 · AI Engineering Internship, AI Development Part

Company Website · samsungfire.com

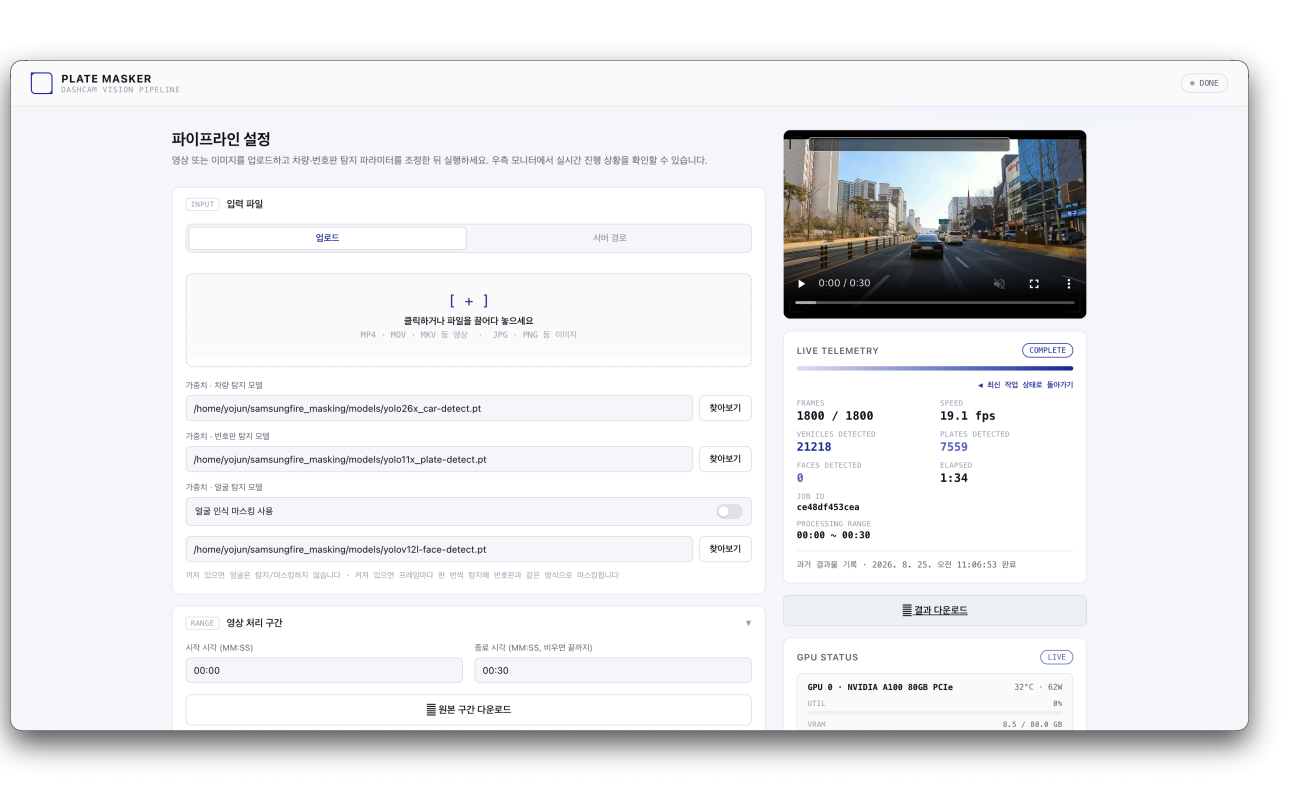
Automatic License Plate Masking

Website · masking.yojun.dev

Built a module that automatically detects and masks vehicle license plates in dashcam video.

- Collected dashcam and plate-image datasets from AI Hub and Roboflow Universe. To capture small plates, I first detected vehicles, enlarged each vehicle crop, and applied a separate plate detector.
- Fine-tuned the model with blur and downscaling, achieving **99.0% recall on the image evaluation set**.
- Combined optical flow, batch inference, FP16, parallel video-segment processing, and FFmpeg to achieve **27+ frames per second in the development GPU environment**.
- Built a FastAPI service for uploads, processing options, progress tracking, previews, and downloads.

Technologies: Python, PyTorch, Ultralytics YOLO, OpenCV, FFmpeg, FastAPI.



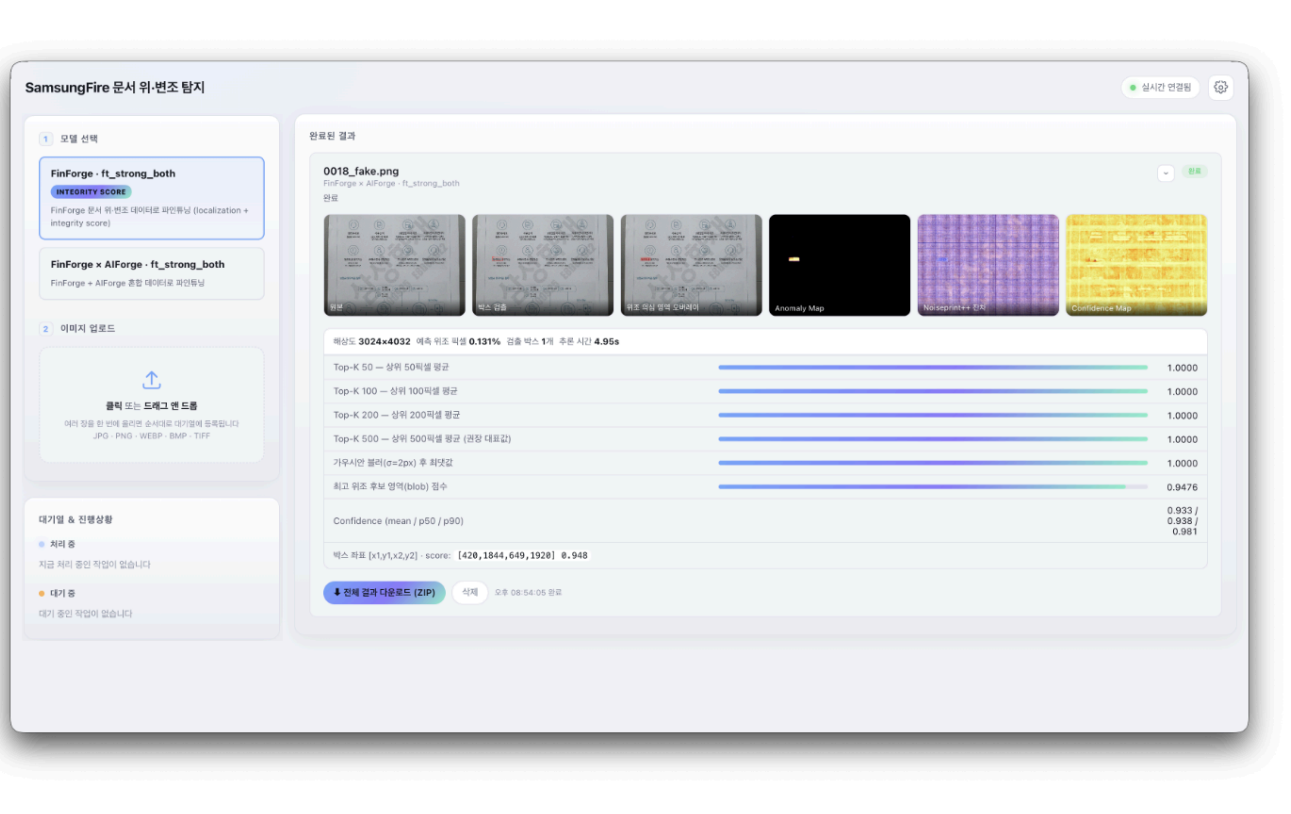
Generative-AI Document Forgery Detection — FinForge / TruFor

Detection Service · forge.yojun.dev Dataset Viewer · docforgery.knpu.re.kr

Developed a module that detects documents with generative-AI edits and identifies the altered regions, working with another intern.

- Built **6,000 original/forged images using eight image-generation models** and added 3,000 independent genuine documents for training and evaluation.
- Fine-tuned TruFor (CVPR 2023), connecting region localization to document-level classification and raising accuracy from **about 55% to 93% on our test set**.
- Evaluated blur, noise, JPEG recompression, and external data. Additional training with image degradation addressed weak conditions.
- Delivered a detection service and a dataset viewer, and organized reports, code, and datasets so the company could continue the research.

Technologies: Python, PyTorch, TruFor, image-generation APIs, FastAPI.



---

2. PCSS — Faculty Candidate Search

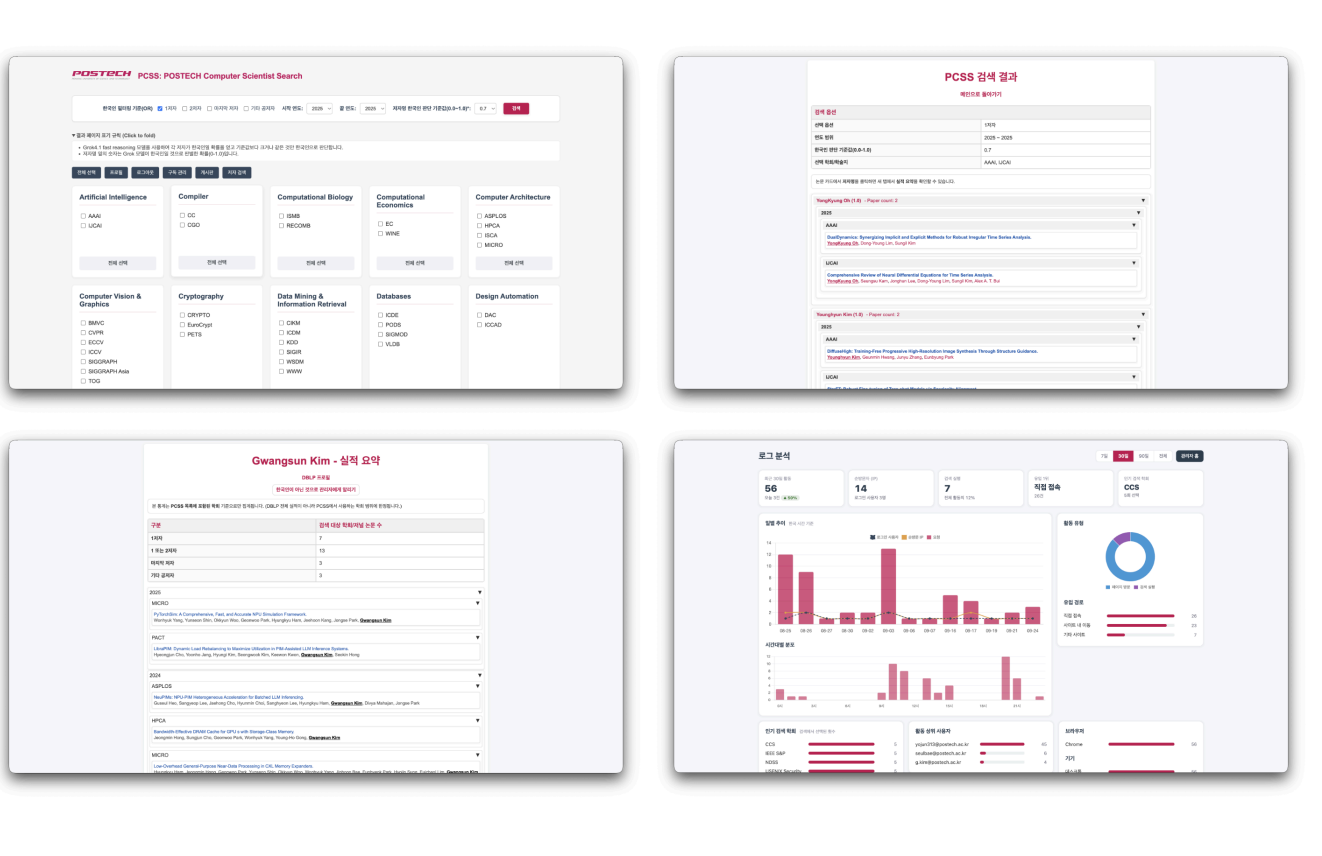
Initial development: Jan 2025 – Mar 2026 · Commissioned by the POSTECH CSE Department

Website · pcss.postech.ac.kr GitHub · yojun313/pcss

Built PCSS to help POSTECH faculty find potential Korean faculty candidates and review their publication records without manually browsing conference pages.

- **Publication database:** Parsed DBLP XML into MongoDB, storing titles, venues, years, authors, and researcher identifiers. Queries by venue, year, and author position reduced hours of manual browsing to seconds against the prepared database.
- **Grok-based name scoring:** Used the Grok API to assign a 0–1 Korean-name likelihood score, persisted results in MongoDB, and cached them in memory. Users can adjust the threshold to broaden or narrow the candidate pool.
- **Search workflow:** Combined conference, date-range, name-score, and author-position filters. Added progress updates, cancellation, shareable search conditions, and CSV/Excel export.
- **Researcher profiles:** Displayed affiliations, links, publication lists, author-role counts, annual trends, venue distributions, and frequent coauthors.
- **Researcher rankings:** Aggregated publication counts across all registered venues or a selected field, with date, author-role, score-threshold, and newcomer filters. Cached repeated aggregations.
- **Monthly updates:** Ran publication and author updates in separate processes to reclaim memory between stages, then refreshed the in-memory author and name-score caches.
- **Subscriptions:** Saved users' conference, date-range, and score preferences and emailed matching new papers after data updates.

Technologies: Python, FastAPI, MongoDB, Motor/PyMongo, Jinja2, JavaScript, XML parsing, Grok API, SSE, multiprocessing.



In Progress: NameClassifier — Korean-Name Scoring

Developing a local model that scores Korean-name and romanization patterns to reduce PCSS's cost and dependence on commercial LLM APIs.

- Combined real names from Wikidata and a sports API with generated data, varying romanization, spacing, and name order to build **approximately 4.7 million name records**, including spelling variants.
- Added **LoRA adapters and a scalar scoring head to Qwen3-0.6B-Base**. Experimented with score imbalance and loss weighting and trained with multi-GPU DDP.
- At a 0.5 threshold, achieved **98.19% recall, 97.19% precision, and 97.69% F1** on the 30,550-record benchmark; further model development is ongoing.
- Built a FastAPI inference and evaluation interface with SQLite history, threshold adjustment, error inspection, ROC/PR views, and experiment-specific data and configuration tracking.

Technologies: Python, PyTorch, Transformers, PEFT/LoRA, Qwen3, DDP, BF16, FastAPI, SQLite, JSONL.

---

3. Commeet — Meeting Scheduling and Record Management

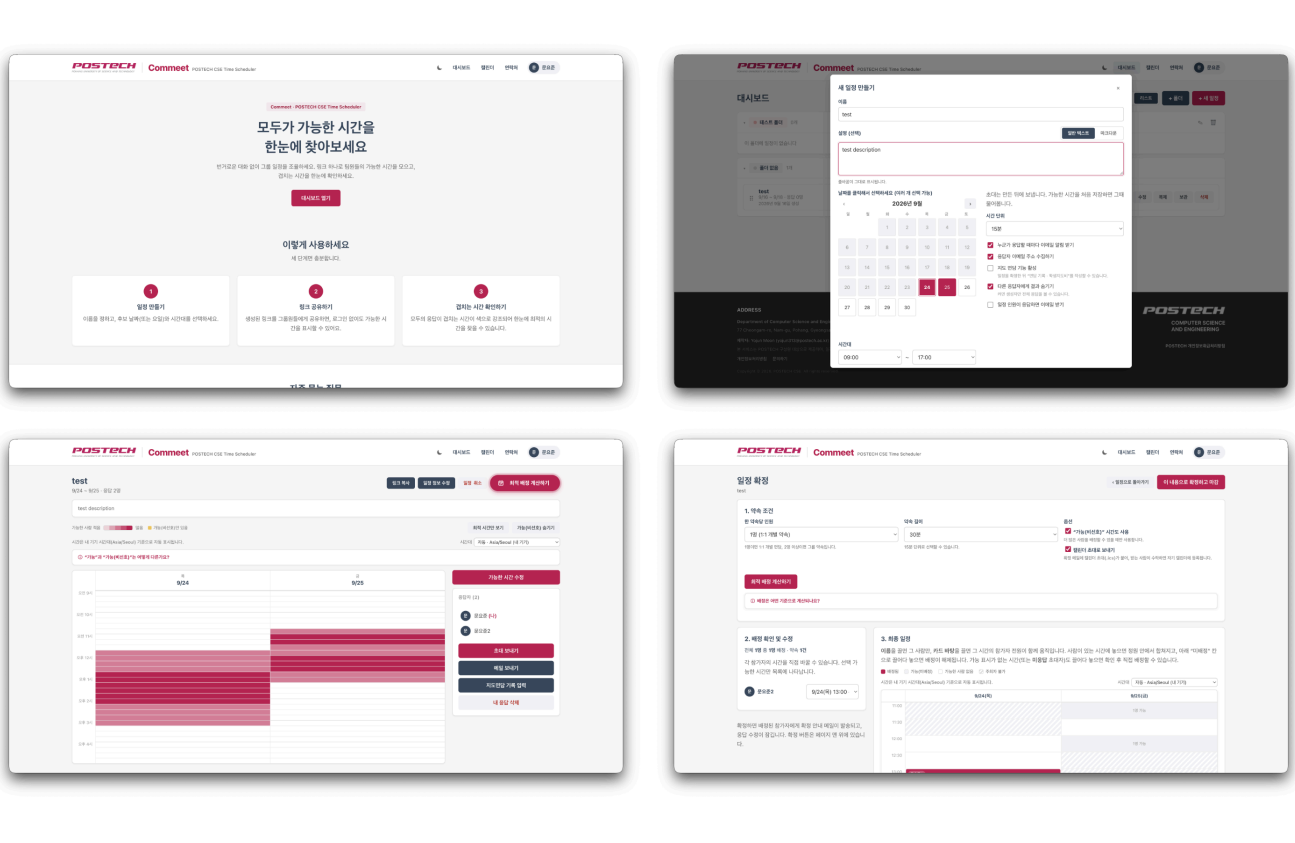
Aug 2026 – Present · Service for the POSTECH CSE Department

Website · commeet.postech.ac.kr GitHub · yojun313/Commeet

Developed Commeet to automatically assign consultation times from participants' availability and manage the records afterward. The service has reached **100+ cumulative users**.

- Used **min-cost max-flow** to maximize the number of scheduled participants, with preferred-slot costs and regrouping heuristics to consolidate sessions.
- Added guest responses, private availability responses, email-domain verification, and Google Calendar integration.
- Supported consultation records, signatures, and receipts, with Fernet encryption for stored records.

Technologies: Python, FastAPI, MongoDB, Motor, Jinja2, JavaScript, MCMF, Google Calendar API, Fernet.



---

4. Additional Academic Projects

HAiV Lab / ACCIDENTOR

Lab Website · HAiV

Jan 2026 – Feb 2026 · Undergraduate Research

Contributed to research on an AI system that analyzes dashcam footage to understand traffic accidents and estimate fault ratios. The work connected local VLM analysis with embedding-based retrieval from a vector database.

Technologies: Python, PyTorch, VLMs, embeddings, vector database.

POPO / PoApper

Jan 2026 – Feb 2026 · Campus Service Development

Website · popo.poapper.club

Contributed to the backend of POPO, a campus web service operated by POSTECH's student developer club, PoApper.

Technologies: TypeScript, NestJS.

---

5. Korean National Police University — Research Software

Nov 2023 – Present · Big Data Research Lab

Research Systems · knpu.re.kr/systems GitHub · github.com/yojun313/knpu

Built a research platform that supports data collection, analysis, visualization, and document generation. Extended the workflows from desktop tools to web services and connected them to tool-using agents, evidence-based AI interpretation, and job monitoring.

Research Server Operations

Solely responsible for operating and maintaining the lab's **two research servers**, including Linux environments, MongoDB databases, local LLM serving with vLLM, and web services through Nginx.

Server	CPU	GPU	RAM	Storage
Server 1	Intel Xeon Gold 6442Y	NVIDIA A4000 16GB	128GB	2TB
Server 2	Intel Xeon Gold 5315Y	NVIDIA A100 80GB × 2	377GB	20TB

Technologies: Linux (Ubuntu Server), MongoDB, vLLM, Nginx, PM2.

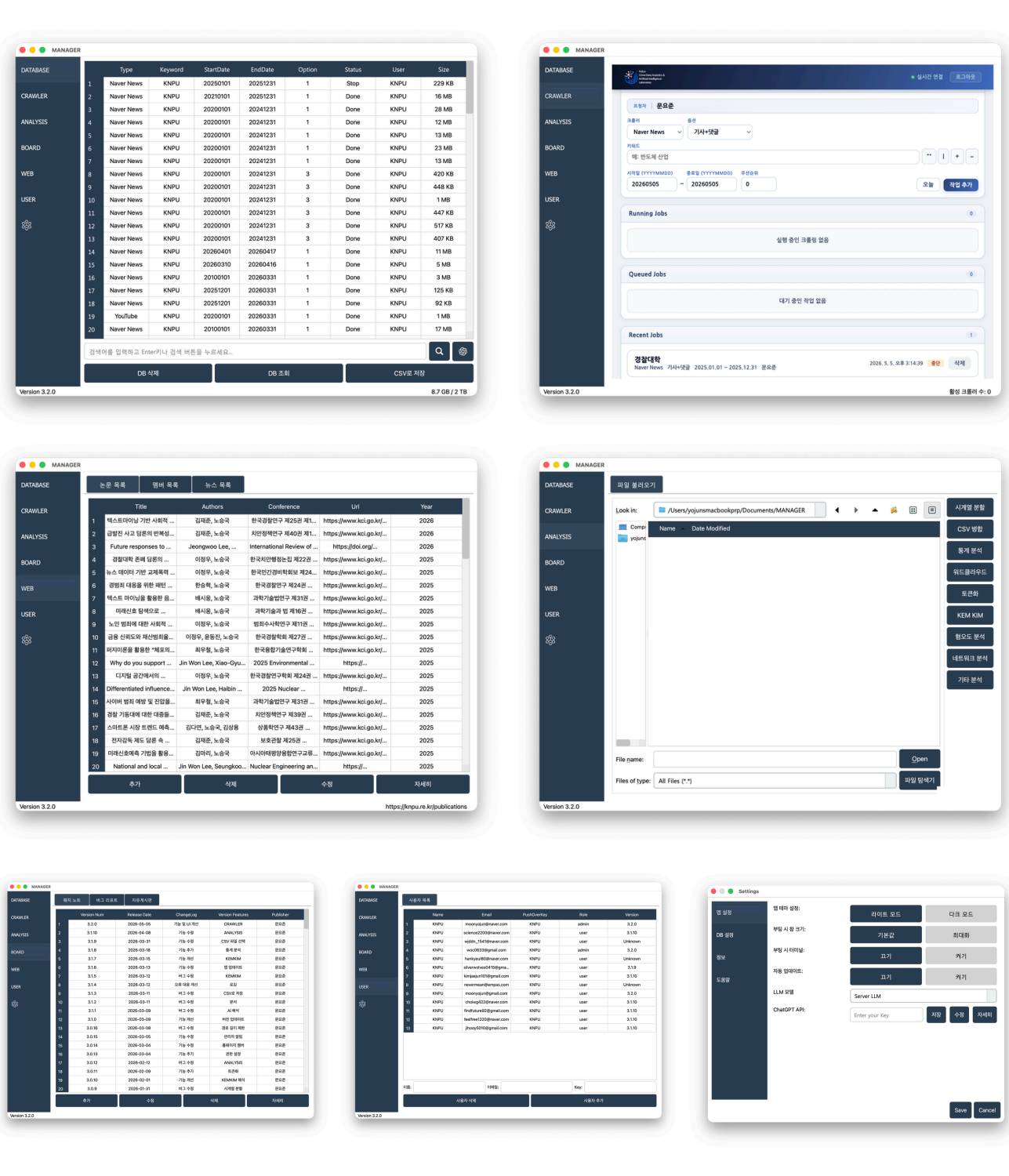
MANAGER — Research Data and Shared Operations

Website · manager.knpu.re.kr GitHub · KNPU

Developed an integrated management tool for browsing research data, running collection and analysis tools, and managing users.

- Started with a Qt desktop application and extended the research functions to web services.
- Centralized service ports, domains, and development/production settings. Recorded per-job progress, logs, heartbeats, and failure states.
- Built shared controls for local LLM/API selection, fallback to a user's API, encrypted API-key storage, and estimated usage tracking.

Technologies: Python, Qt (PyQt/PySide6), FastAPI, MongoDB, PM2, OpenAI-compatible APIs, vLLM.



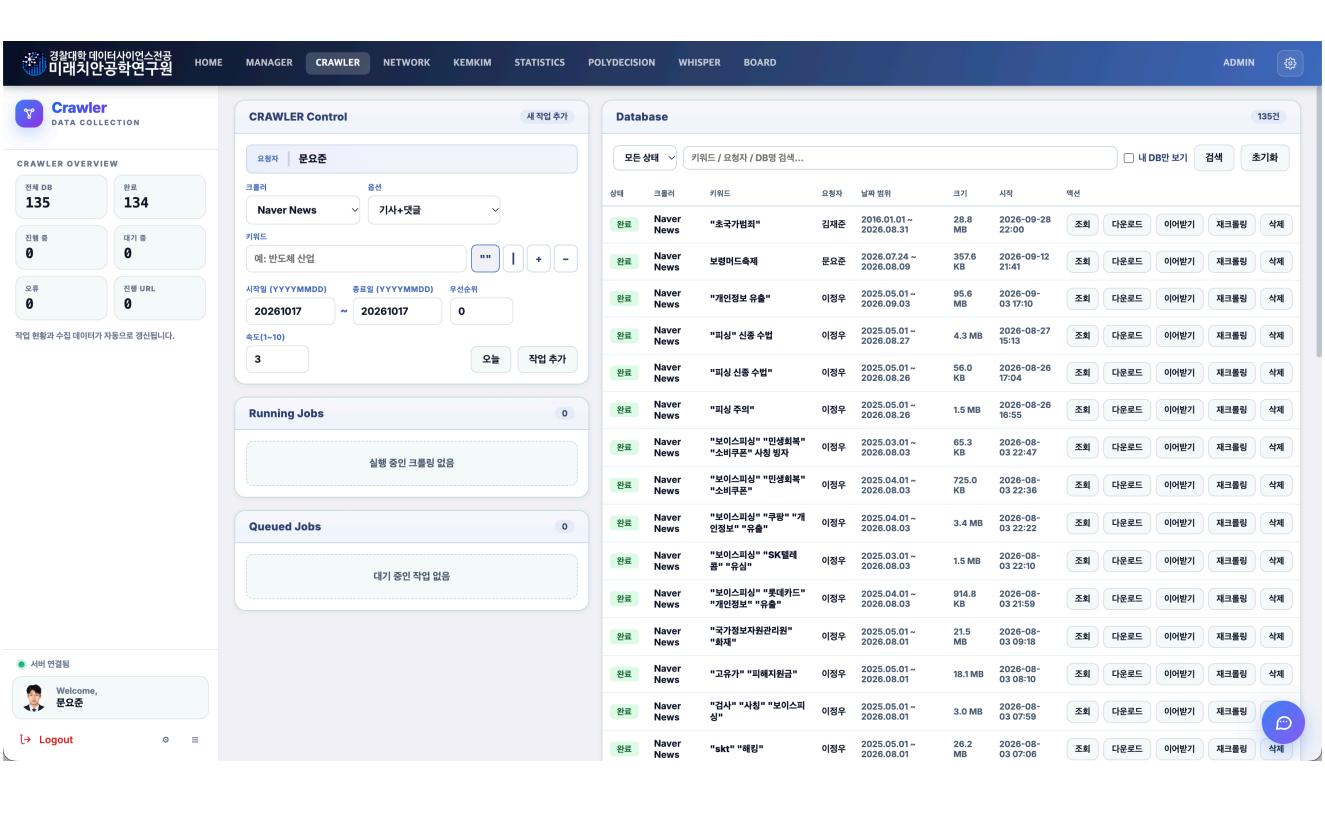
CRAWLER — Asynchronous Data Collection

Website · crawler.knpu.re.kr GitHub · KNPU

Implemented a crawler that collects news, blog, cafe, and YouTube data for a selected research topic and period.

- Applied asynchronous I/O and rotating proxy IPs, improving collection speed by approximately **3–4× over the synchronous implementation**.
- Implemented a priority queue, concurrency limits, cancellation, resume, and re-collection, with web-based progress and result monitoring.

Technologies: Python, FastAPI, asyncio, aiohttp, BeautifulSoup, MongoDB.



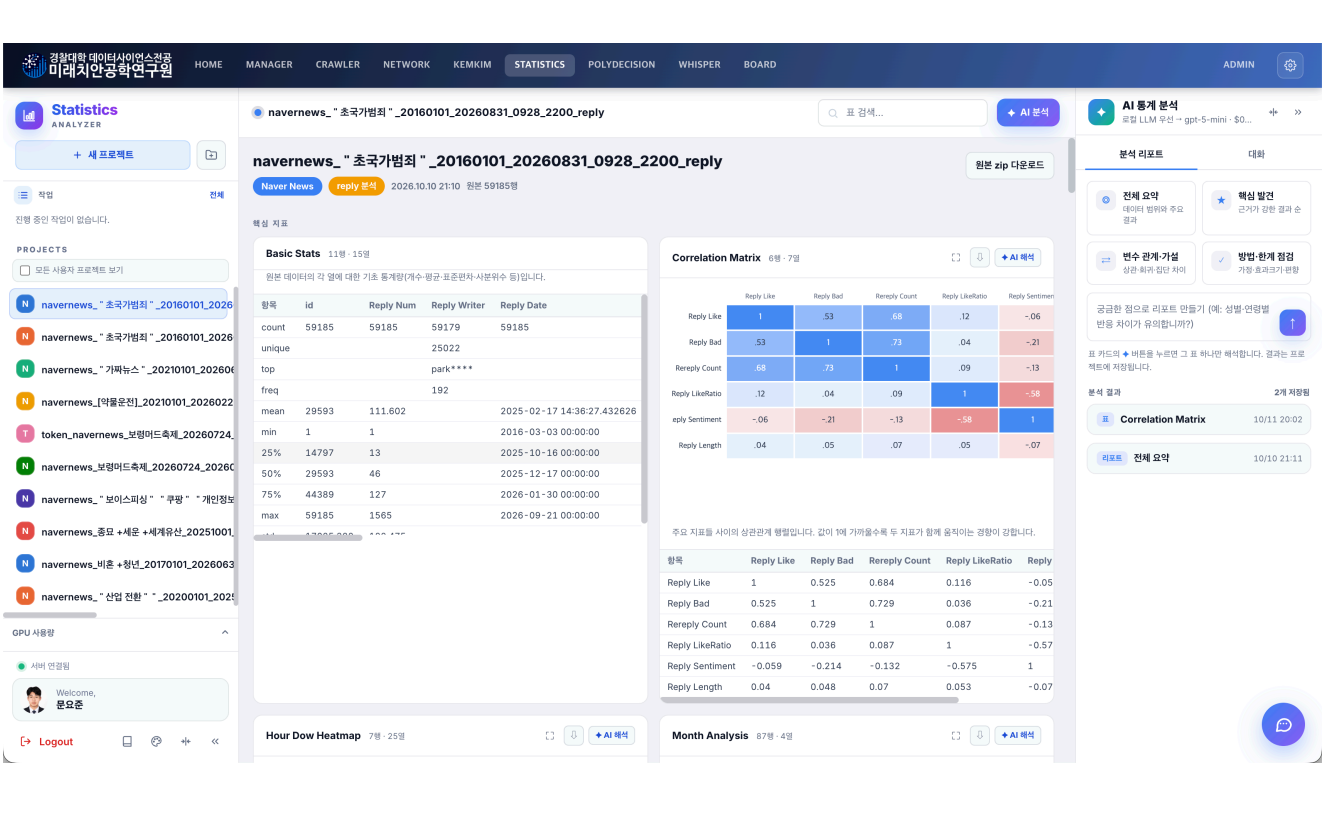
STATISTICS — Statistical Analysis and AI Agent

Website · statistics.knpu.re.kr GitHub · KNPU

Built a service that analyzes research data, visualizes results, and provides AI interpretation and question answering.

- Connected descriptive statistics, correlation, regression, group comparisons, PCA, and clustering to tables and charts.
- Generated **AI analysis reports grounded in aggregate tables**, with server-side checks that compare cited table IDs and numbers against the source data.
- Built a question-answering agent that repeatedly calls **table-search, table-reading, and calculation tools**. Summarized long tables server-side and excluded tables marked as private from AI input.

Technologies: Python, FastAPI, pandas, SciPy, statsmodels, scikit-learn, Matplotlib, Seaborn, LLM APIs.



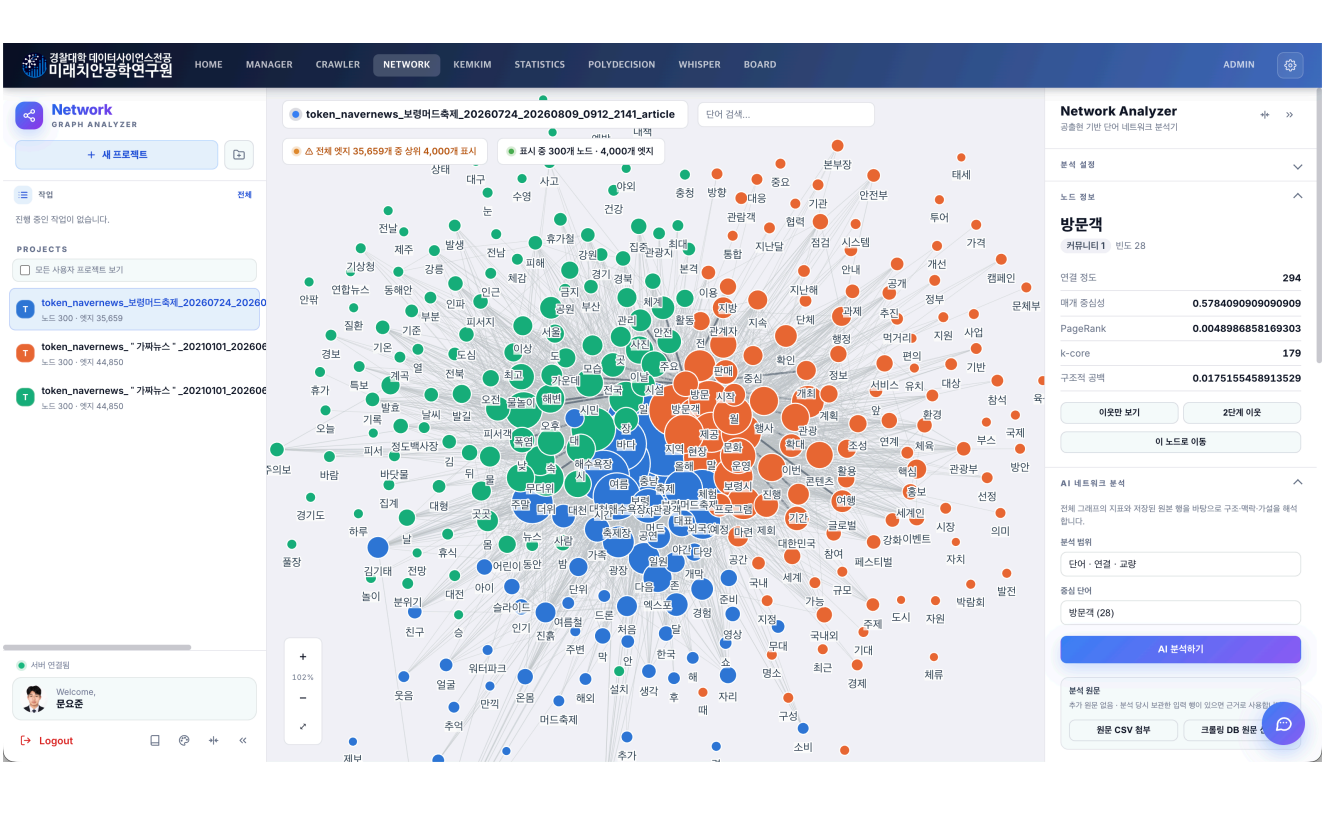
NETWORK — Network Analysis and AI Interpretation

Website · network.knpu.re.kr GitHub · KNPU

Developed a service that visualizes word and entity relationships in text as networks and explains them with AI.

- Added interactive exploration of connections and communities, with **AI analysis for the full graph, individual keywords, word pairs, and communities.**
- Supplied server-computed network metrics and relevant source text to the LLM, linking explanations to supporting material.
- Distinguished URL-matched original evidence from supplementary search results and tracked AI analyses as background jobs.

Technologies: Python, FastAPI, pandas, NumPy, igraph, LLM APIs.



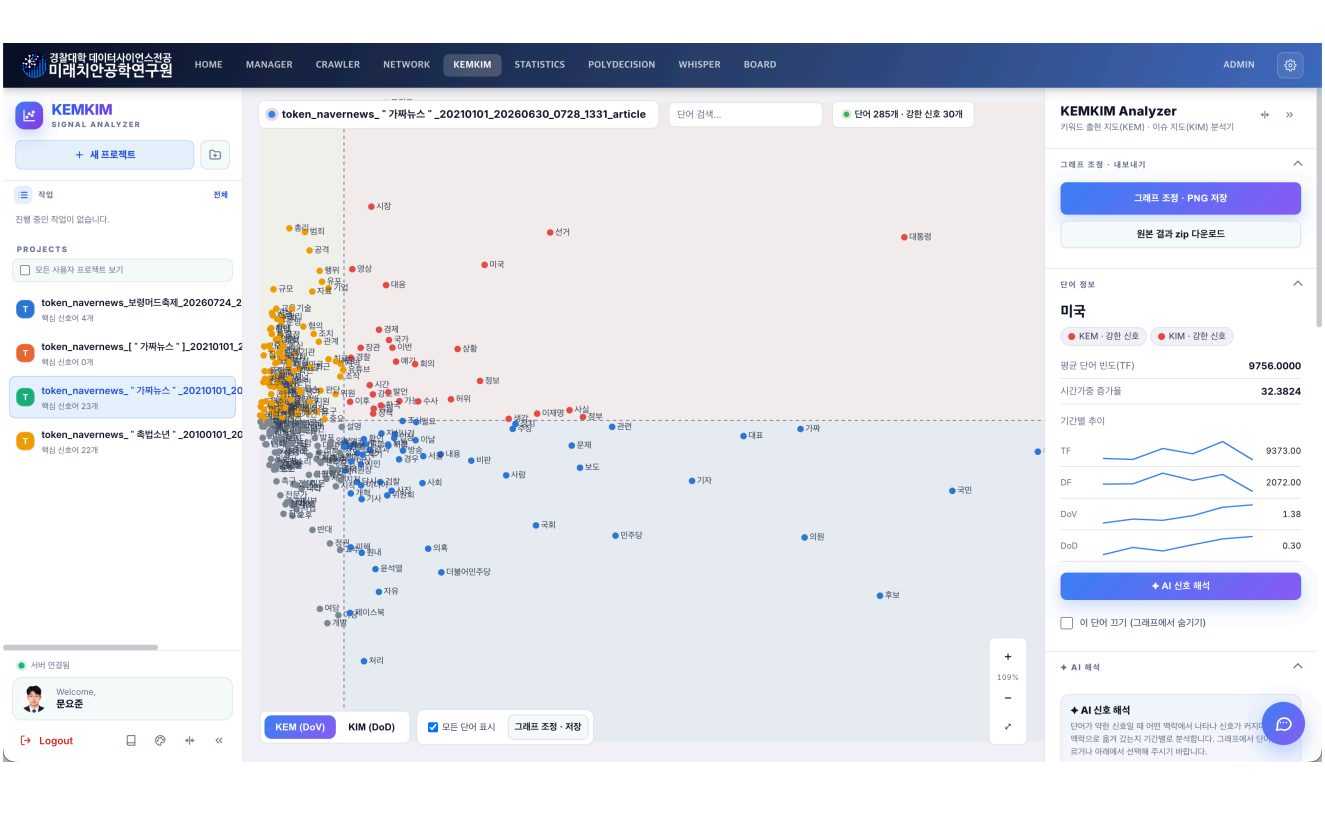
KEMKIM — Keyword Signal Analysis and AI Explanation

Website · kemkim.knpu.re.kr GitHub · KNPU

Implemented a KEM/KIM analysis service that classifies potential future signals from keyword trends and provides AI explanations.

- Implemented the published methodology, calculating DoV/DoD from word frequency, document counts, and time weights, and visualizing signal categories and changes over time.
- Built context from signal transitions, related terms, and representative articles so **AI can interpret the background of observed changes**. Linked evidence IDs in the explanation to the original articles.
- Added adjustable periods, weights, and classification criteria, with result comparison and export.

Technologies: Python, FastAPI, pandas, NumPy, Matplotlib, LLM APIs.

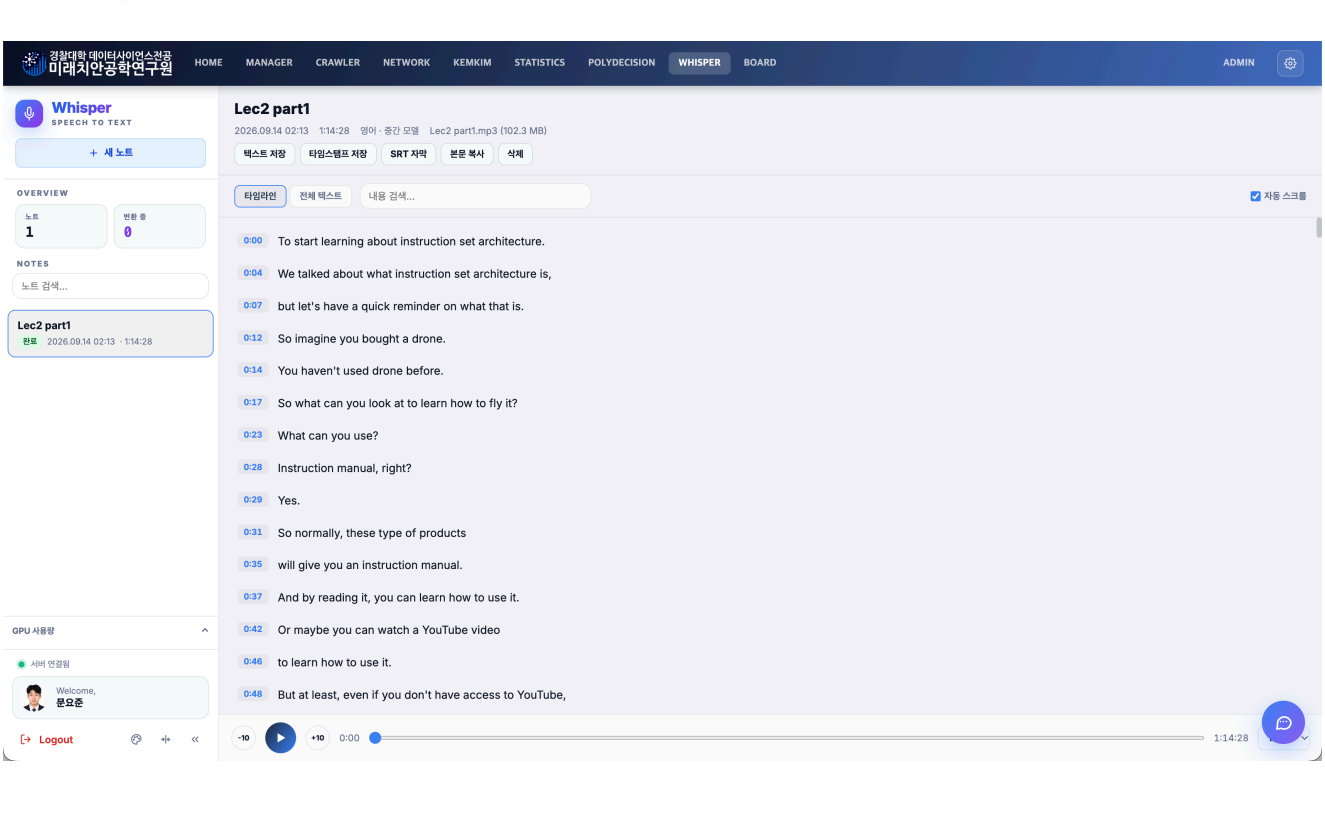


WHISPER — Audio Transcription

Website · whisper.knpu.re.kr GitHub · KNPU

Built a service that converts audio files into text and manages transcription results. It connects to a GPU transcription server, limits concurrent transcriptions, and retries transient connection failures.

Technologies: Python, FastAPI, httpx, faster-whisper, GPU.



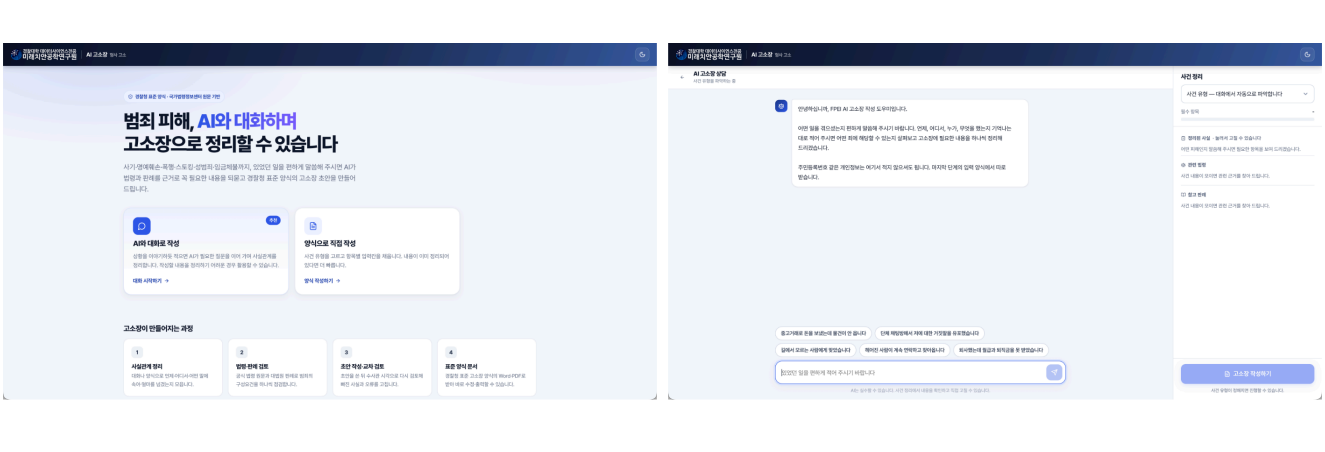
AI Complaint Drafting

Website · complaint.knpu.re.kr GitHub · KNPU

Developed a service that turns a user's incident description into an AI-generated criminal complaint draft and downloadable documents.

- Converted structured JSON output from the LLM into a document template, with Word/PDF generation, previews, and downloads.
- Kept separately entered personal details out of the LLM input. Received a Letter of Appreciation from the Korean National Police University for research software contributions in **April 2024**.

Technologies: Python, FastAPI, LLM APIs, JSON, python-docx, PDF conversion.



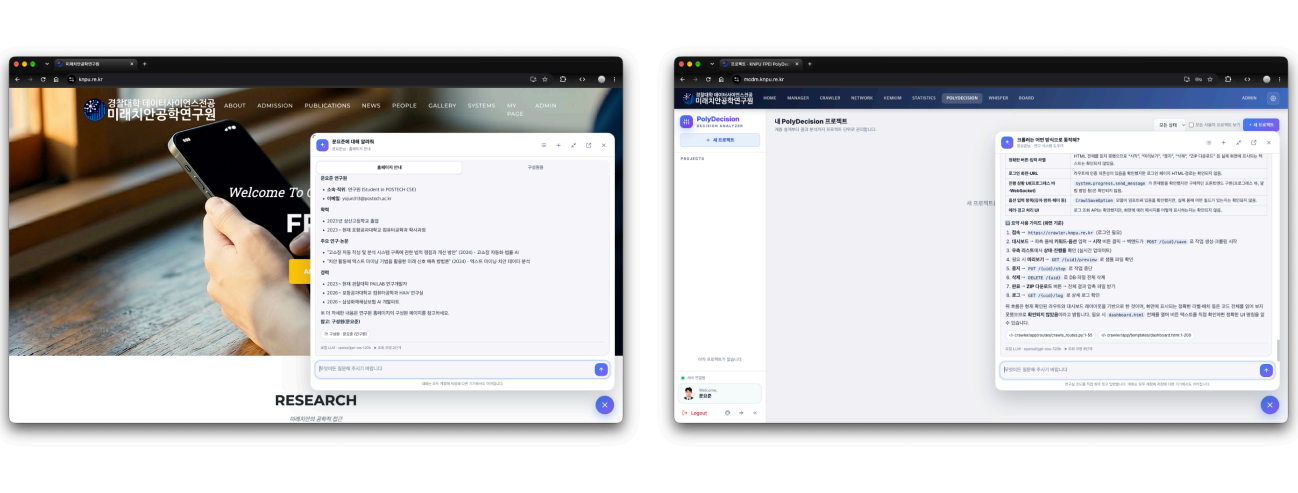
Agent Chatbot — Research Information and Workflow Guidance

Website · knpu.re.kr GitHub · KNPU

Built a chatbot that answers public questions about the lab and guides signed-in researchers through its tools and multi-service workflows.

- Built a custom agent loop that **selects a tool, inspects its result, searches further when needed, and produces an answer**, with JSON tool requests interpreted and executed by the server.
- Connected website documents, member and publication search, service guides, code search, and file-reading tools. Used BM25 retrieval and source inspection to ground responses in actual material.
- Separated public/member tool permissions and added repeated-call detection, time and turn limits, and context-length management.
- Recorded conversation history, progress events, and usage so users could follow the agent's work and status.

Technologies: Python, FastAPI, custom agent loop, JSON tool calling, BM25, asynchronous jobs, status APIs, local LLMs, OpenAI-compatible APIs.



---

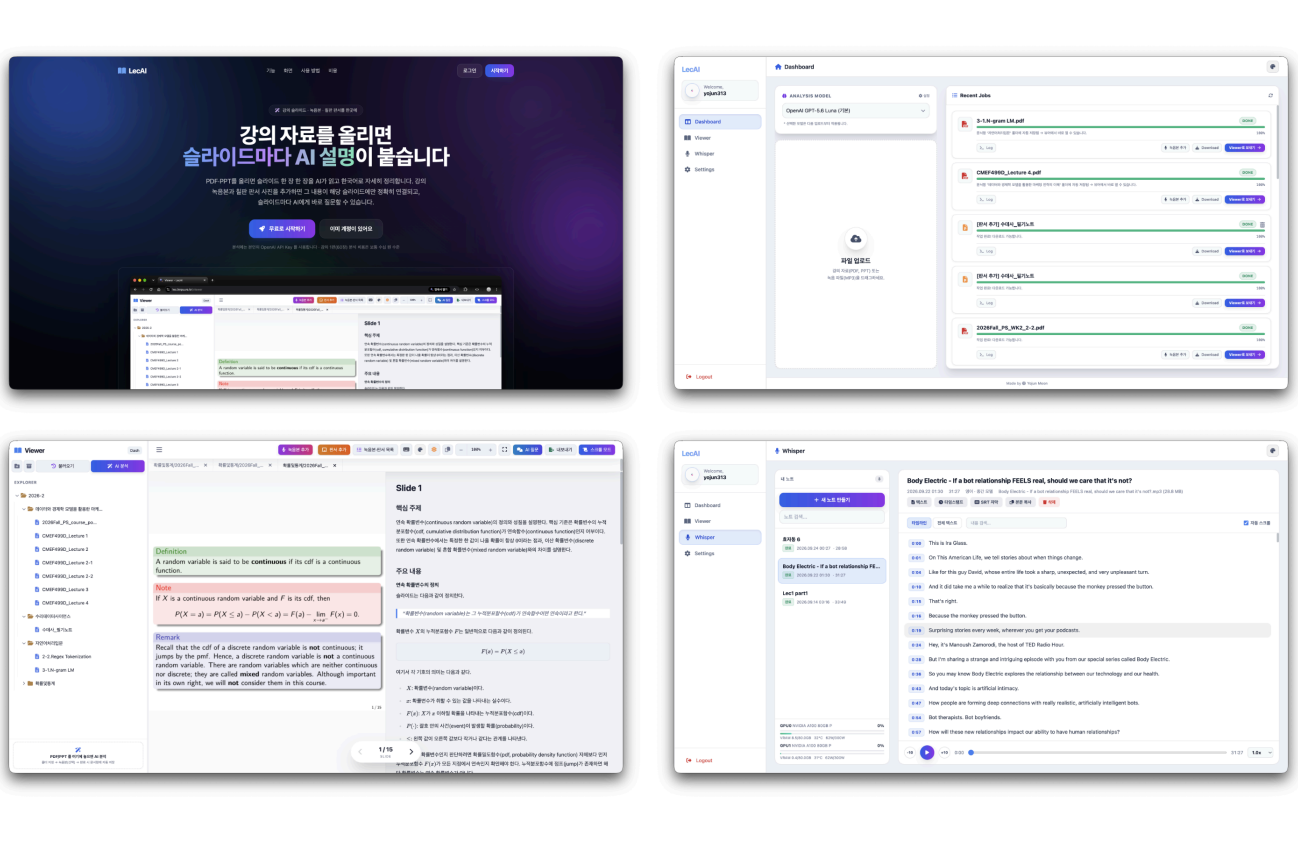
6. LecAI — Study Assistant for Slides, Recordings, and Board Notes

Website · lec.yojun.dev GitHub · github.com/yojun313/LecAI

Developed an AI study assistant that connects lecture materials, recordings, and board notes to slide-level explanations.

- Generated explanations for PDF/PPT slides and connected relevant transcript excerpts and board images using speech transcription and VLM analysis.
- Added a **chatbot for each slide**, using the slide image, its explanation (including transcript excerpts and board notes), and the lecture outline as context. Included conversation history for follow-up questions and streamed replies through SSE.
- Supported local/remote Whisper and API transcription. Added transcript quote checks, mathematical expression conversion to LaTeX, and step-by-step explanations for board notes.
- Managed progress, logs, results, and user ownership. Limited concurrent jobs with a worker pool and semaphore, and marked interrupted jobs as failed after a server restart.

Technologies: Python, FastAPI, MongoDB, Whisper/faster-whisper, LLM/VLM APIs, ThreadPoolExecutor, FFmpeg, LaTeX.



---

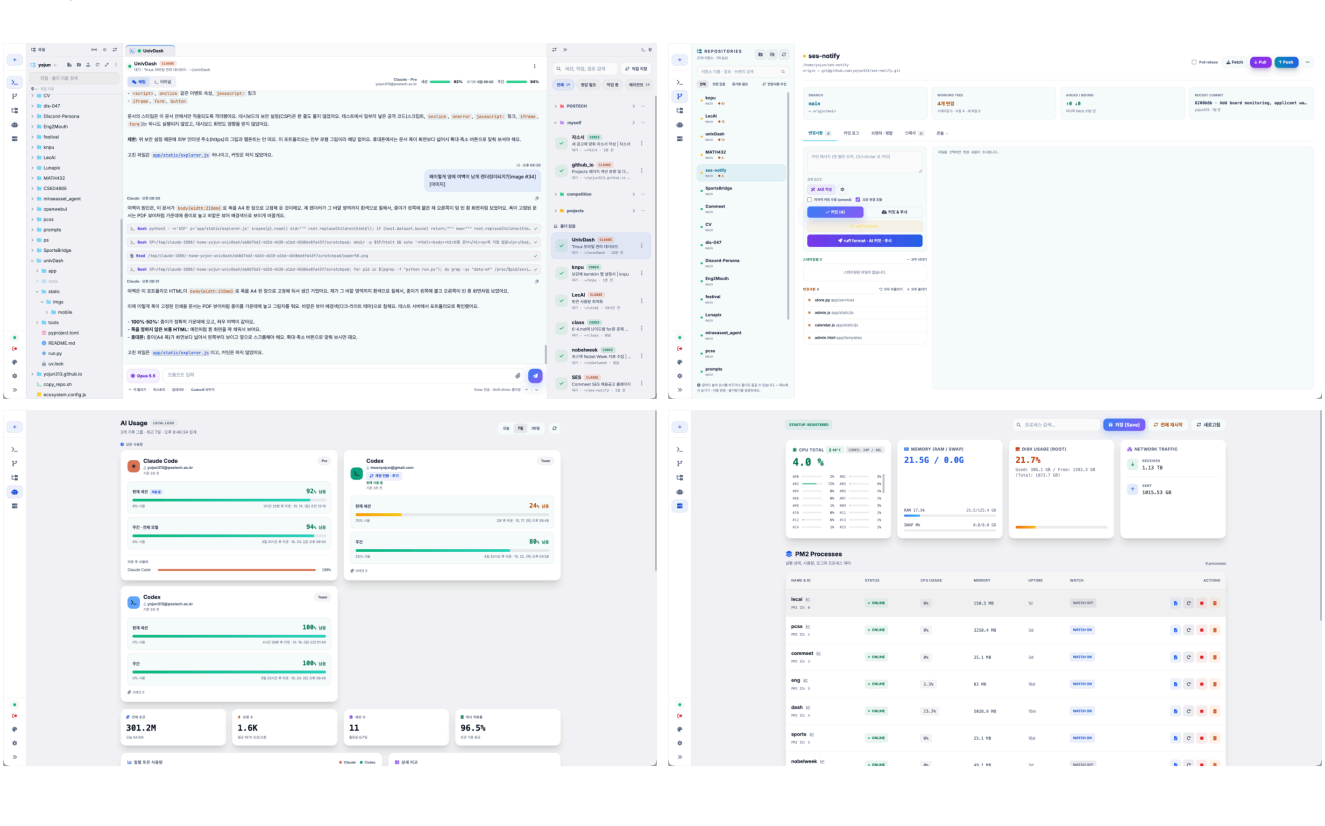
7. UnivDash — Coding Agent and Development Server Dashboard

GitHub · yojun313/UnivDash

Built a web dashboard for monitoring and controlling server-side coding agents and development tools from a PC or phone. **I currently use UnivDash for all my development work**, continuously adding features and improving it through daily use.

- Connected terminals through **tmux, PTY, and WebSocket**. Incrementally parsed actual Claude Code and Codex session logs to show conversation history and distinguish working, awaiting-input, and idle states.
- Added prompt and file submission, file browsing and editing with conflict detection, and PM2/server-resource monitoring.
- Integrated Git operations and AI-generated commit messages. Applied login/session controls, TOTP, and WebSocket Origin validation.

Technologies: Python, FastAPI, Jinja2, JavaScript, WebSocket, tmux, PTY, PM2, Git, LLM APIs, JSON file storage.



---

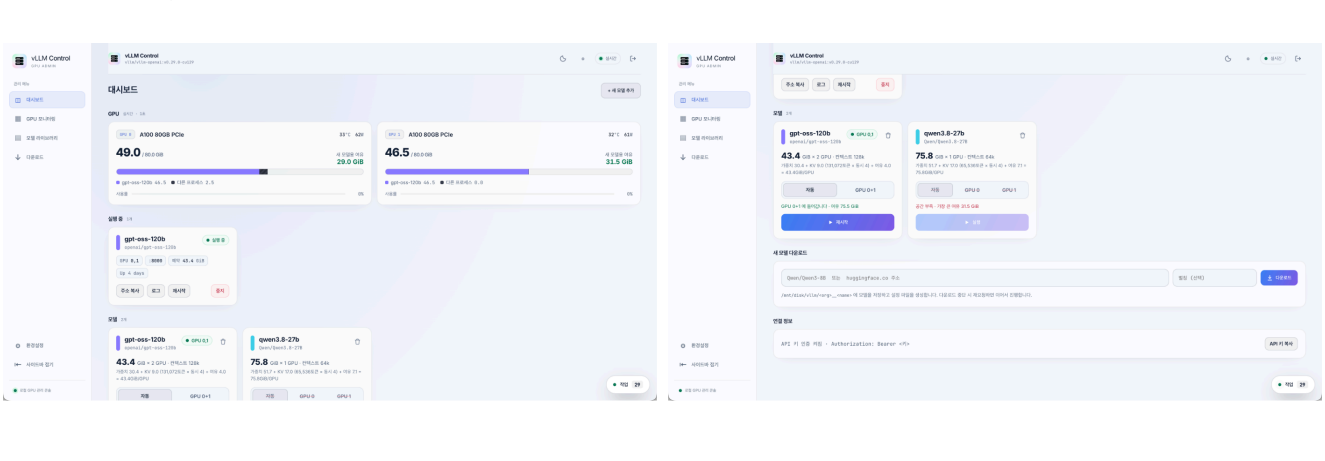
8. vllm-manager — Local LLM Deployment and GPU Resource Management

GitHub · yojun313/vllm-manager

Developed a CLI and web console for deploying local LLMs and managing model status and GPU resources on a GPU server.

- Automated Hugging Face downloads and YAML configuration generation, then launched **vLLM OpenAI-compatible API servers in Docker**.
- Estimated memory requirements from model weights, KV cache, context length, concurrency, and tensor parallelism. Checked actual GPU usage and startup reservations before launching a model.
- Ran downloads and startup as separate processes with persisted job status and logs. Streamed GPU, container, and job status over WebSocket and restricted duplicate requests.

Technologies: Python, Shell, FastAPI, Docker, vLLM, Hugging Face Hub, YAML, NVIDIA GPU/nvidia-smi, WebSocket, PyJWT.



---

9. Eng2Mouth — Real-Time AI English Conversation

Website · eng.yojun.dev GitHub · github.com/yojun313/eng2mouth

Built a service for practicing spoken English through real-time AI voice calls, with post-call assessment and expressions to review.

- Connected **OpenAI Realtime over WebRTC** and **Gemini Live over WebSocket**, issuing temporary browser connection tokens and unifying provider-specific connections into a common call workflow.
- Connected the **`end_call` tool call** to actual call termination and saved structured post-call feedback, CEFR levels, and useful expressions.
- Managed call history, usage, and cost. Added conversation partners and scenarios, expression review, and a dashboard for tracking practice.

Technologies: Python, FastAPI, MongoDB, Jinja2, JavaScript, WebRTC, WebSocket, OpenAI Realtime API, Gemini Live API.